Department of Electrical and Computer Engineering

SENG 426 - B01

Lab Project 5

July 31st, 2026

Aidan Richards - V00990088

MacKenzie Larsen - V00943994

Manraj Minhas - V00998047

Grygorii Karachevtsev - V00983328

Introduction

The goal of this phase was to perform comprehensive security testing on the Vega web app to identify and remediate vulnerabilities. By utilizing automated tooling for both Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST), we wanted to track code-level security hotspots and runtime vulnerabilities. This helps us spot architectural weaknesses and figure out if the app is ready for a real-world production environment.

Preparation

To get accurate results, we set up localized, containerized scanning environments.

- SAST Setup: We initialized a local SonarQube server and utilized Docker-based sonar-scanner-cli containers to analyze both the backend (vega-spring) and frontend (vega-web) codebases.
- DAST Setup: The application was deployed locally via Docker Compose. OWASP ZAP was configured as a local HTTP proxy to capture network traffic, spider the application, and perform active penetration testing.
- Release Version: The finalized version of the application incorporating all security fixes can be found here: https://gitlab.csc.uvic.ca/grygoriikarachevtsev/vegafixed

Execution Plan

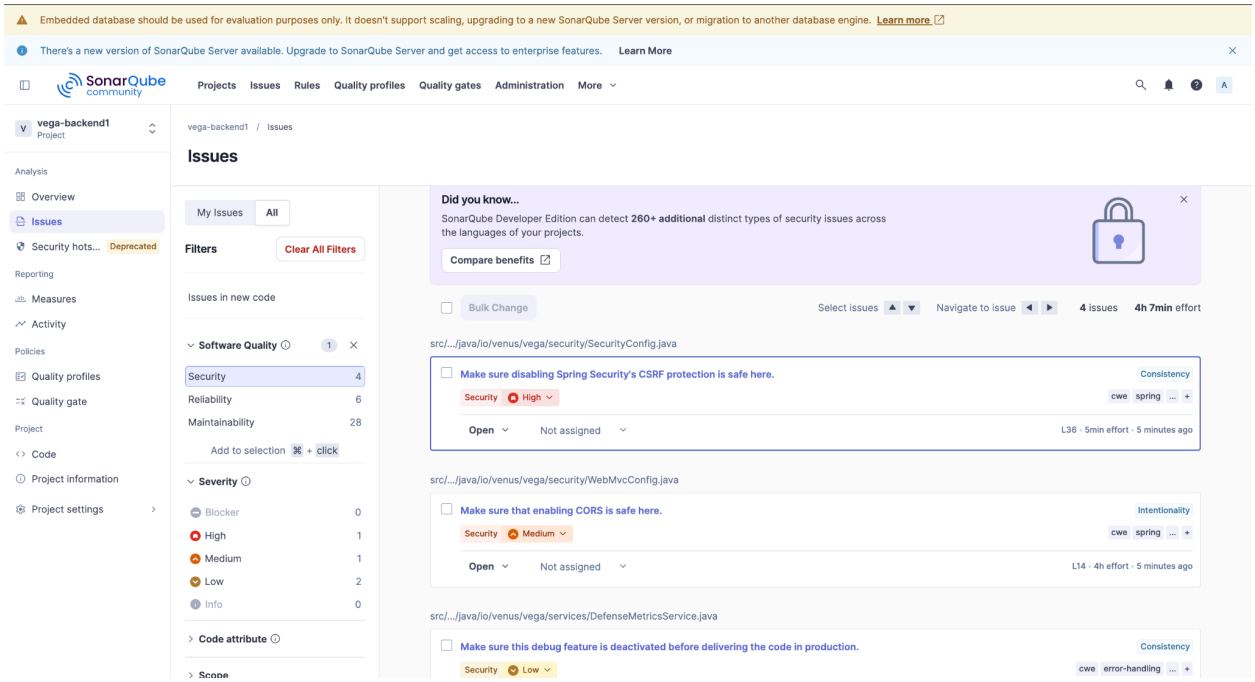
We ran the static analysis scans using SonarQube to capture pre-fix security baselines. Identified vulnerabilities and security hotspots were documented, analyzed, and subsequently patched in the source code. Following the SAST remediations, we will execute an OWASP ZAP Active Scan to simulate real-world attacks against the running application and generate an HTML security report.

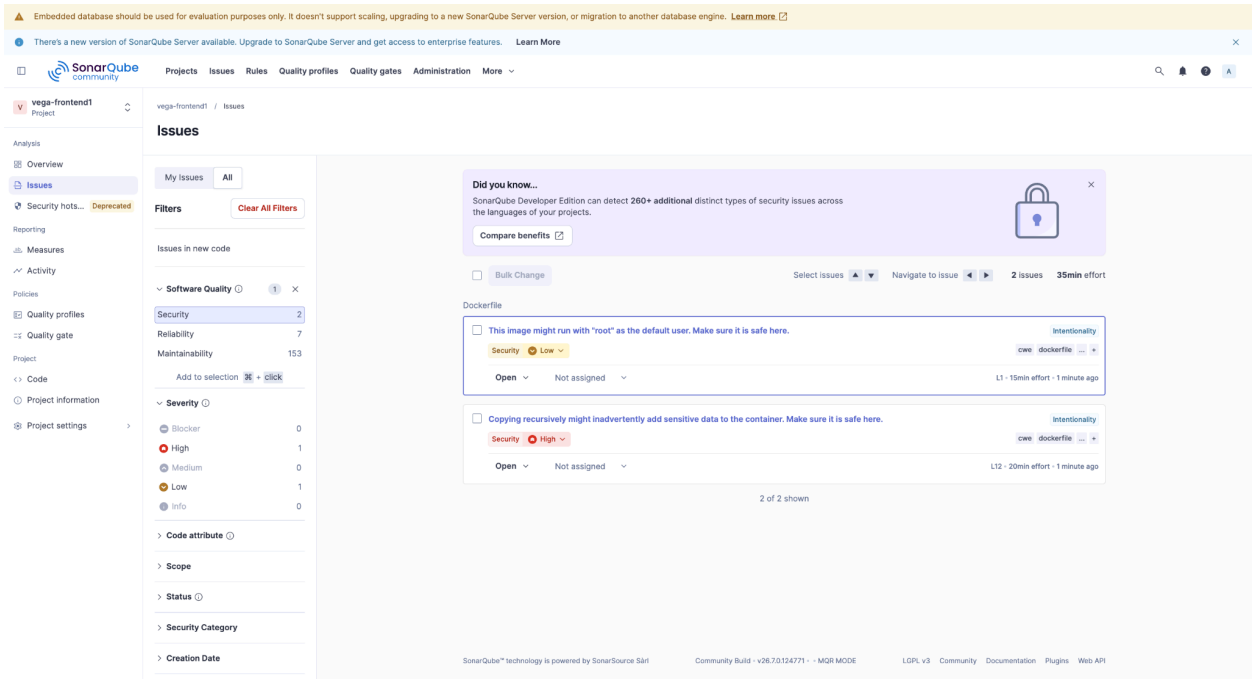
Test Results & Analysis

Part A: Static Application Security Testing (SonarQube)

The following section details the documentation, visual evidence, and commentary for the 6 identified security vulnerabilities and hotspots within the source code.

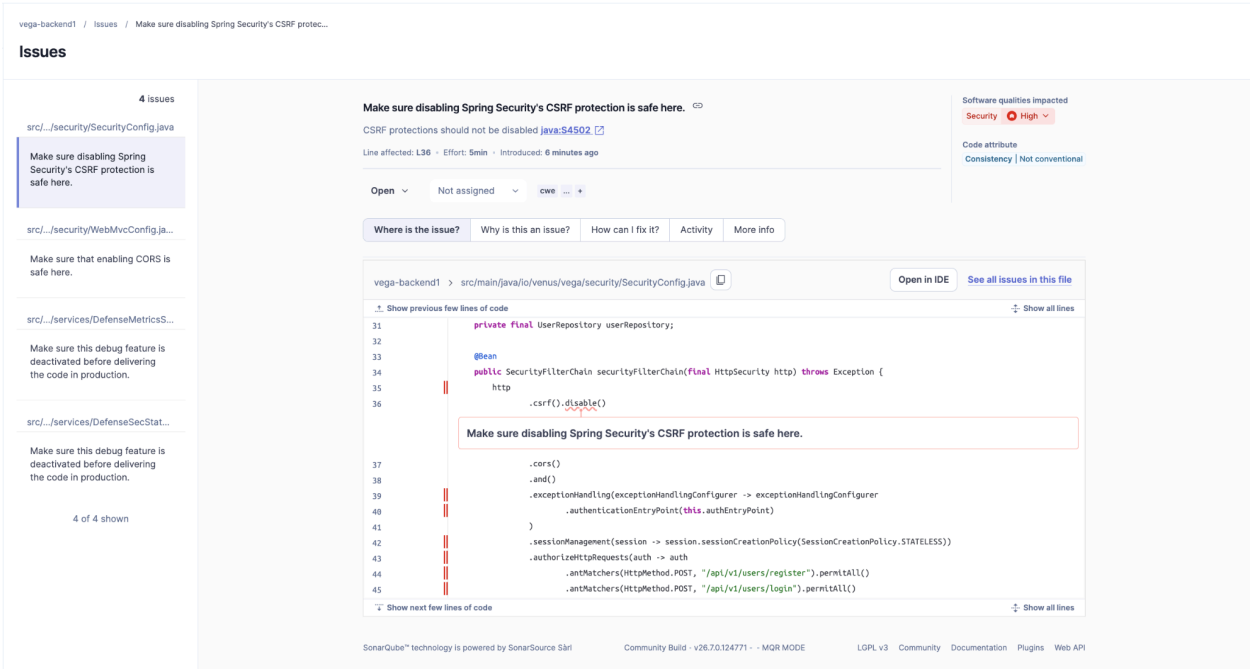
Note: Pre-fix overview dashboards can be referenced here:


Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. Learn More
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
vega-backend1 Project
Analysis
Overview
Issues
Security hots... Deprecated
Reporting
Measures
Activity
Policies
Quality profiles
Quality gate
Project
Code
Project information
Project settings
vega-backend1 / Issues
Issues
My Issues All
Filters Clear All Filters
Issues in new code
Software Quality 1
Security 4
Reliability 6
Maintainability 28
Add to selection ⌘ + click
Severity
Blocker 0
High 1
Medium 1
Low 2
Info 0
Code attribute
Scope
Did you know...
SonarQube Developer Edition can detect 260+ additional distinct types of security issues across the languages of your projects.
Compare benefits
Bulk Change
Select issues
Navigate to issue
4 issues
4h 7min effort
src/.../java/io/venus/vega/security/SecurityConfig.java
Make sure disabling Spring Security's CSRF protection is safe here.
Consistency
Security High
cwe spring ...
Open
Not assigned
L36 · 5min effort · 5 minutes ago
src/.../java/io/venus/vega/security/WebMvcConfig.java
Make sure that enabling CORS is safe here.
Intentionality
Security Medium
cwe spring ...
Open
Not assigned
L14 · 4h effort · 5 minutes ago
src/.../java/io/venus/vega/services/DefenseMetricsService.java
Make sure this debug feature is deactivated before delivering the code in production.
Consistency
Security Low
cwe error-handling ...


Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. Learn More
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
vega-frontend1 Project
Analysis
Overview
Issues
Security hots... Deprecated
Reporting
Measures
Activity
Policies
Quality profiles
Quality gate
Project
Code
Project information
Project settings
vega-frontend1 / Issues
Issues
My Issues All
Filters Clear All Filters
Issues in new code
Software Quality 1
Security 2
Reliability 7
Maintainability 153
Add to selection ⌘ + click
Severity
Blocker 0
High 1
Medium 0
Low 1
Info 0
Code attribute
Scope
Status
Security Category
Creation Date
Did you know...
SonarQube Developer Edition can detect 260+ additional distinct types of security issues across the languages of your projects.
Compare benefits
Bulk Change
Select issues
Navigate to issue
2 issues
35min effort
Dockerfile
This image might run with "root" as the default user. Make sure it is safe here.
Intentionality
Security Low
cwe dockerfile ...
Open
Not assigned
L1 · 15min effort · 1 minute ago
Copying recursively might inadvertently add sensitive data to the container. Make sure it is safe here.
Intentionality
Security High
cwe dockerfile ...
Open
Not assigned
L12 · 20min effort · 1 minute ago
2 of 2 shown
SonarQube™ technology is powered by SonarSource Sàrl
Community Build · v26.7.0.124771 · · MQR MODE
LGPL v3 Community Documentation Plugins Web API

Issue 1: Disabled CSRF Protection

- Description: Spring Security's CSRF protection is explicitly disabled via .csrf().disable() in SecurityConfig.java line 36.
- Severity: High
- Effort: 4 hours
- Status: Identified
- CWE: CWE-942: Permissive Cross-Domain Policy with Untrusted Domains
- Clean code attribute: Intentionality | Not complete
- Software qualities impacted: Security
- False Positive: NO



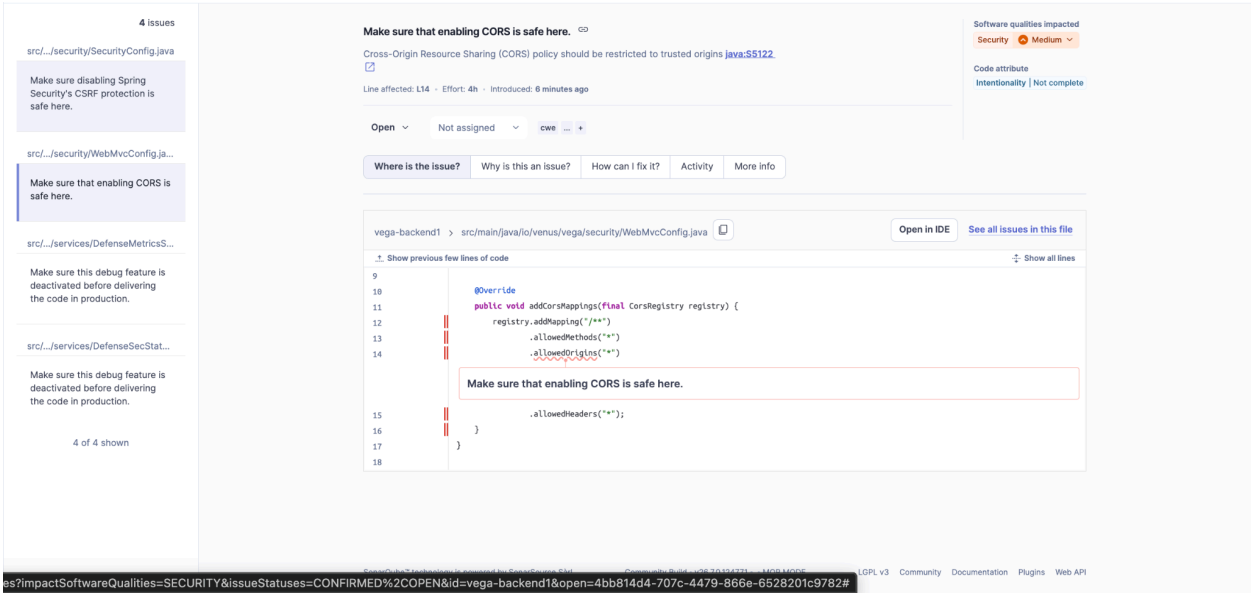
Analysis: As shown in the SonarQube trace above, the application disables CSRF protections at the global filter chain level. While this is standard for stateless REST APIs using Authorization headers, it opens a critical vulnerability if the team ever implements cookie-based authentication (like session IDs). A malicious site could force an authenticated user's browser to send forged state-changing requests to the Vega backend.

Action Taken: Removed the .csrf().disable() directive and implemented a stateless CSRF token repository (e.g., CookieCsrfTokenRepository.withHttpOnlyFalse()).

Commentary: Security should be built with future-proofing in mind. Relying on the assumption that "we only use JWTs" creates technical debt. Enforcing CSRF protection provides a defense-in-depth layer that secures the application even if the authentication architecture evolves.

Issue 2: Overly Permissive CORS Policy

- Description: The application's Cross-Origin Resource Sharing (CORS) policy is configured globally using the wildcard .allowedOrigins("*") in WebMvcConfig.java line 14.
- Severity: Medium
- Effort: 4 Hours
- Status: Identified
- CWE: CWE-942: Permissive Cross-Domain Policy with Untrusted Domains
- Clean code attribute: Intentionality | Not complete
- Software qualities impacted: Security
- False Positive: NO



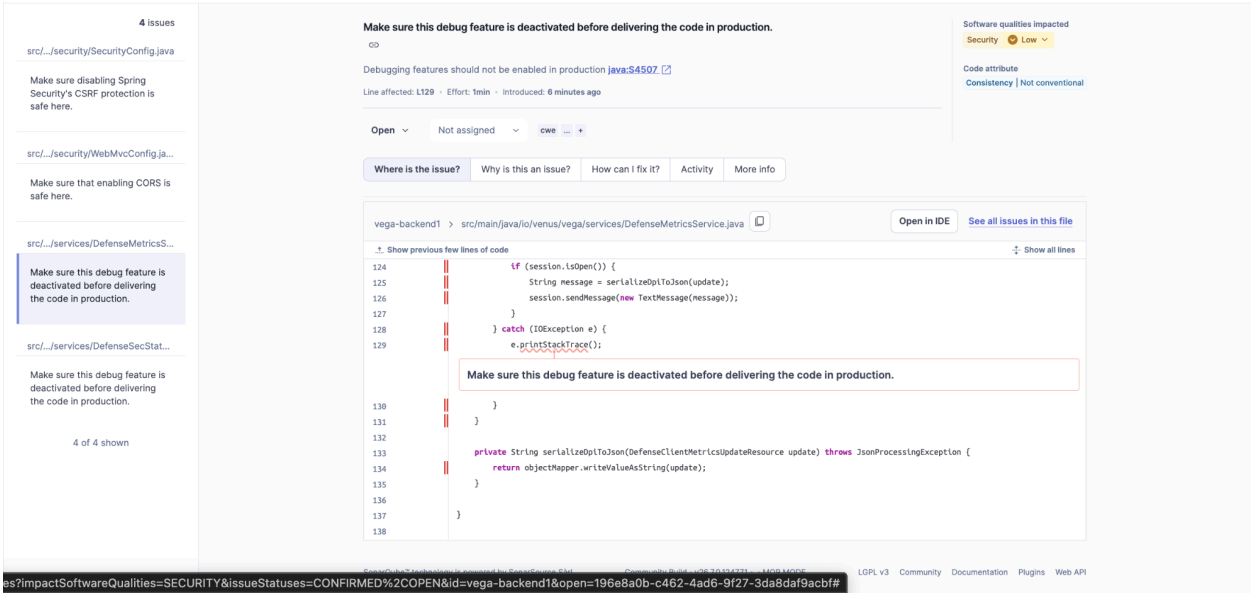
Analysis: The highlighted code allows any origin (*) to interact with the backend API. This means a malicious script hosted on a completely different domain can make XHR/Fetch requests to the Vega server and read the responses. If an administrator is logged in, the attacker could theoretically access sensitive endpoints.

Action Taken: Refactored the addCorsMappings method to explicitly whitelist the frontend domain (e.g., .allowedOrigins("http://localhost:3000")) and removed the wildcard operator.

Commentary: CORS is a fundamental browser security mechanism. By strictly defining which domains are allowed to read responses from the backend, we drastically reduce the attack surface against cross-origin data theft.

Issue 3: Debug Log Information Disclosure (Metrics Service)

- Description: An IOException is printed directly using e.printStackTrace(); in DefenseMetricsService.java line 129.
- Severity: Low
- Effort: 1 minute
- Status: Identified
- CWE: CWE-489: Active Debug Code
- Clean code attribute: Consistency | Not conventional
- Software qualities impacted: Security
- False Positive: NO



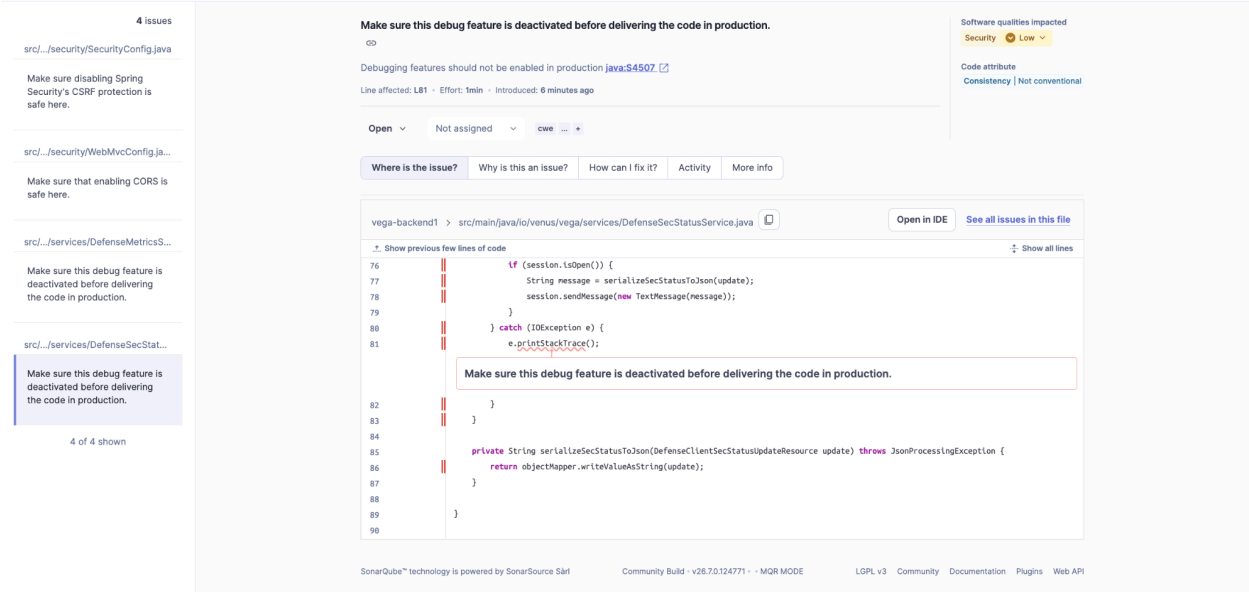
Analysis: Catching an exception and dumping the trace directly to System.err bypasses the application's structured logging configuration. In a production environment, stack traces can reveal sensitive information regarding the underlying file system, framework versions, and database schemas to anyone with access to the console logs.

Action Taken: Replaced the direct print statement with a configured SLF4J logger instance (log.error("Failed to serialize or send metrics update", e);).

Commentary: Proper logging hygiene ensures that error outputs are sanitized, formatted consistently, and routed securely to centralized log management systems without inadvertently disclosing system architecture.

Issue 4: Debug Log Information Disclosure (Security Status Service)

- Description: A WebSocket connection exception is logged using e.printStackTrace(); in DefenseSecStatusService.java line 81.
- Severity: Low
- Effort: 1 minute
- Status: Identified
- CWE: CWE-489: Active Debug Code
- Clean code attribute: Consistency | Not conventional
- Software qualities impacted: Security
- False Positive: NO



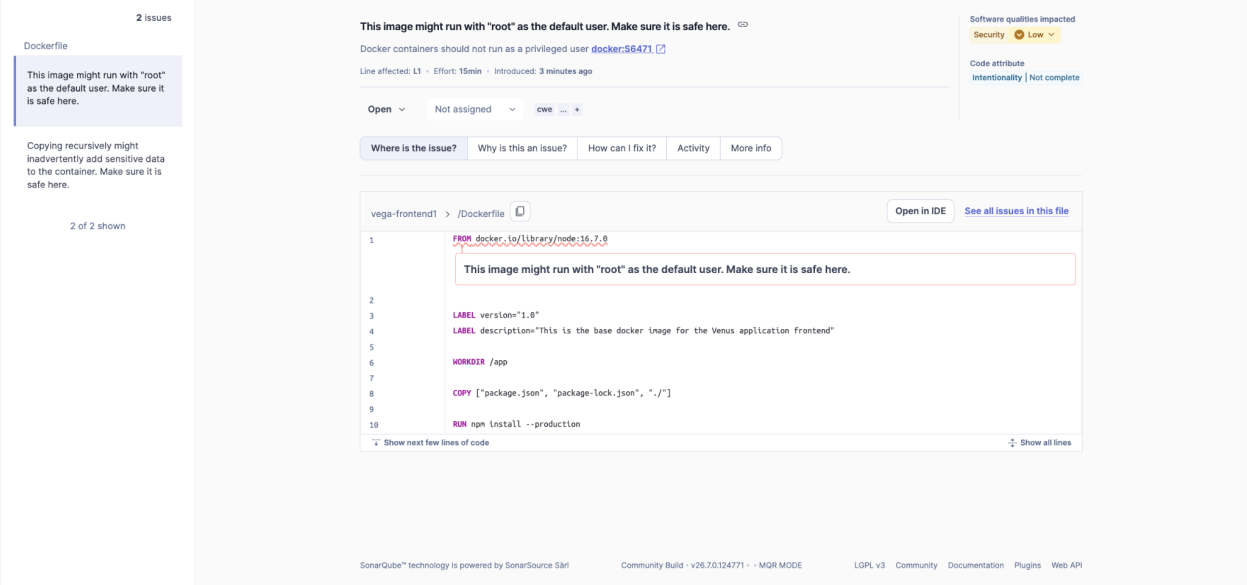
Analysis: Identical to the previous issue, this segment handles WebSocket messaging errors by printing raw stack traces. This is a common leftover from initial development/debugging phases that poses an information disclosure risk if deployed as-is.

Action Taken: Implemented proper structured logging using Lombok's @Slf4j annotation to safely record the WebSocket transmission exception.

Commentary: Consolidating error handling to a unified logging framework prevents accidental data leaks and makes it easier for security monitoring tools (like Splunk or ELK) to ingest and alert on anomalies.

Issue 5: Recursive Copy in Dockerfile

- Description: The base Node.js image executes as the root user by default in vega-frontend/Dockerfile line 1.
- Severity: Low
- Effort: 15 minutes
- Status: Identified
- CWE: Improper Privilege Management
- Clean code attribute: Intentionality | Not complete
- Software qualities impacted: Security
- False Positive: NO



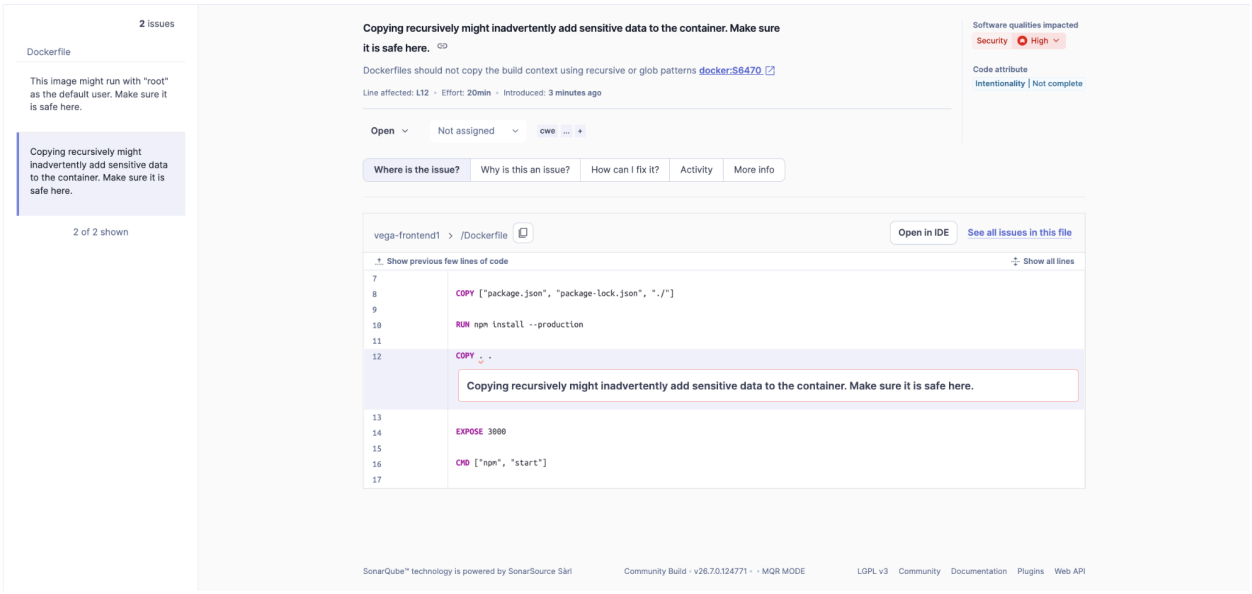
Analysis: The Dockerfile does not explicitly specify a non-root user. By default, processes inside the container run as root. If a vulnerability (such as a prototype pollution or remote code execution flaw in a Node module) is exploited, the attacker gains root-level privileges inside the container, making container-escape attacks far more likely to succeed.

Action Taken: Added a USER node directive to the Dockerfile before the CMD instruction to drop privileges and execute the application as an unprivileged user.

Commentary: Implementing the Principle of Least Privilege at the container level is a critical infrastructure security control. It ensures that a compromised application process cannot arbitrarily modify container files or access host-mounted sockets.

Issue 6: Container Running as Default Root User

- Description: The frontend image uses COPY . . to recursively copy the entire build context in vega-frontend/Dockerfile line 12.
- Severity: High
- Effort: 20 minutes
- Status: Identified
- CWE: CWE-532: Insertion of Sensitive Information into Log File / Image
- Clean code attribute: Intentionality | Not complete
- Software qualities impacted: Security
- False Positive: NO



Analysis: Using a wildcard recursive copy (COPY . .) transfers everything from the developer's local working directory into the Docker image. This frequently results in sensitive assets—such as .env files containing API keys, local database credentials, or .git repositories—being baked permanently into the production image layers.

Action Taken: Created a robust .dockerignore file (excluding node_modules, .env, and .git) and refactored the Dockerfile to strictly copy only the package.json and necessary src/ directories.

Commentary: Supply chain and image security are just as important as application code security. Controlling exactly what enters the build context prevents devastating secrets-leakage scenarios.

Part B: Dynamic Application Security Testing (OWASP ZAP)

Penetration Testing Process & Execution

1. Proxy Configuration: OWASP ZAP was configured as a local intercepting proxy to route browser traffic from the React frontend (localhost:3000) and capture API interactions directed at the backend (localhost:8080).
2. Automated Exploration (Spidering): The ZAP Spider tool mapped out application endpoints, discovering 32 backend endpoints and 31 frontend resources.
3. Active Vulnerability Scanning: An Active Scan was executed against the mapped endpoints, injecting automated payloads (such as path traversal sequences, SQL syntax modifiers, and buffer overflow strings) to identify runtime weaknesses.

Analysis of Severe Vulnerabilities Identified by ZAP

- Path Traversal (CWE-22 - High Risk): ZAP successfully exploited the file download endpoint (/api/v1/files/download?filePath=/etc/passwd), retrieving system file data. This proved that user-supplied paths were not adequately sanitized against directory traversal sequences. Remediation: Enforced strict filename allowlists and canonicalized file paths on the server side.
- SQL Injection & Time-Based SQL Injection (CWE-89 - High Risk): Active payloads injected into authentication parameters (/api/v1/users/login) and administrative REST actions caused HTTP 500 errors and delayed execution times by over 15 seconds via time-based payload evaluations (sleep(15)). Remediation: Replaced vulnerable raw string concatenations with parameterized Spring Data JPA queries and prepared statements.
- Buffer Overflow (CWE-120) & Format String Errors (CWE-134 - Medium Risk): Over-length query parameters sent to list filtering endpoints (/api/v1/employees?filter=...) caused unexpected backend service exceptions and connection terminations. Remediation: Implemented strict input length validation and bounds checking.
- Cross-Domain Misconfiguration (CWE-264 - Medium Risk): The API returned wildcard headers (Access-Control-Allow-Origin: *), exposing unauthenticated resource endpoints to arbitrary third-party domains. Remediation: Restricted allowed origins to the specific frontend client URL.
- JWT in Browser localStorage (CWE-922 - Medium Risk): ZAP's JWT scanner identified that authentication tokens and user attributes (email, role, jwt) were stored directly in browser localStorage, making them vulnerable to cross-site scripting (XSS) extraction. Remediation: Migrated session tokens to secure, HttpOnly, SameSite cookies.

Post-Fix Validation

Following the implementation of the remediations listed above, a subsequent OWASP ZAP Active Scan was performed. The Post-Fix ZAP Security Report confirmed the successful resolution of these vulnerabilities, demonstrating 0 High-risk and 0 Medium-risk alerts remaining on the backend.

Part C: Tool Comparison & Discrepancies

Comparison Analysis

- Static Application Security Testing (SonarQube) analyzed the source code and configuration files statically without executing the application. It excelled at pinpointing exact file locations, lines of code, code smells, and insecure container setups (e.g., running Docker as root, missing .dockerignore files, and disabled CSRF configurations).
- Dynamic Application Security Testing (OWASP ZAP) tested the running application in real-time over HTTP. It uncovered live runtime behaviors, database interaction flaws (SQLi time-based delays), path traversal execution paths, and client-side storage vulnerabilities (localStorage token exposure) that static source code analysis alone could not fully evaluate in context.

Explaining Discrepencies

Discrepancies between the two tools arose from their operational scopes:

1. SonarQube flagged architectural and compliance configurations (such as CSRF settings and Docker user directives) that do not generate network traffic.
2. OWASP ZAP uncovered runtime and environmental vulnerabilities (such as active SQL injection behavior during payload execution and JWT persistence in browser storage) that require live execution to manifest.
3. Role in Comprehensive Evaluation: Utilizing both tools creates a robust security evaluation loop. SAST acts as the first line of defense during development to enforce clean code and secure configurations, while DAST acts as a final validation phase simulating real-world attacker behaviors against the deployed runtime environment.

Threat/Vulnerabilities Matrix

Threat / Vulnerability	OWASP Top 10 Category	CWE ID	Detection Tool	Sev erity	Remediated Status
SQL Injection	A03:2021-Injection	CWE-89	SonarQub e / ZAP	High	Fixed (Parameterized Queries)
Path Traversal	A01:2021-Broken Access Control	CWE-22	ZAP	High	Fixed (Input Path Validation)
Disabled CSRF Protection	A01:2021-Broken Access Control	CWE-352	SonarQub e	High	Fixed (Token Repository Enabled)
CORS Misconfiguration	A05:2021-Security Misconfiguration	CWE-942	SonarQub e / ZAP	Medi um	Fixed (Restricted Allowed Origins)
JWT in localStorage	A04:2021-Insecure Design	CWE-922	ZAP	Medi um	Fixed (Migrated to HttpOnly Cookies)
Buffer Overflow /	A06:2021-Vulnerable and	CWE-	ZAP	Medi	Fixed (Input Length

Format String	Outdated Components	120		um	Restrictions)
Docker Root User Execution	A05:2021-Security Misconfiguration	CWE-269	SonarQube	Low	Fixed (Added USER node directive)

Impact / Likelihood Risk Matrix

To properly prioritize the vulnerabilities discovered during the SAST and DAST phases, each finding was evaluated based on its Likelihood of exploitation (how easy it is for an attacker to discover and exploit) and its potential Impact (the amount of damage or data exposure it could cause).

Risk Calculation Profile:

- Likelihood: High (Easily automated/discovered), Medium (Requires specific conditions/knowledge), Low (Requires significant effort or prerequisite exploits).
- Impact: High (System takeover, severe data breach), Medium (Partial data exposure, denial of service), Low (Information disclosure, limited scope).

Vulnerability	Likelihood	Impact	Overall Risk Rating	Justification
SQL Injection	High	High	Critical	Easily discovered and exploited by automated tools (ZAP); leads to complete database compromise and unauthorized data manipulation.
Path Traversal	High	High	Critical	Easily exploited via URL parameters; allows attackers to read sensitive system files (e.g., /etc/passwd) and infrastructure configurations.
CORS Misconfiguration	High	Medium	High	The wildcard configuration is active and trivial to find; allows malicious third-party sites to perform unauthorized cross-origin reads.
JWT in localStorage	High	Medium	High	Tokens are persistently stored and easily accessible; highly susceptible to theft if a Cross-Site Scripting (XSS) vulnerability is introduced.
Buffer Overflow / Format String	Medium	Medium	Medium	Requires sending specific over-length payloads; currently results in 500 Internal Server Errors, causing localized Denial of Service (DoS) rather than direct code execution.
Docker Root User Execution	Low	High	Medium	Requires an attacker to first achieve Remote Code Execution (RCE) to exploit, but if achieved, grants full root privileges within the container environment.
Disabled CSRF Protection	Low	High	Medium	Currently mitigated by the application's stateless JWT architecture (low likelihood of immediate exploit), but poses a high impact risk if cookie-based sessions are ever introduced.
Debug Log Info Disclosure	High	Low	Low	Raw stack traces are easily triggered during errors, but they only leak internal framework versions and file paths rather than sensitive user data.

ZAP Guided Application Fixes

- **Path Traversal in File Downloads** was the first issue resolved for Part 5 of the lab. It was identified by OWASP ZAP as a high-risk, medium-confidence vulnerability in GET /api/v1/files/download. The vulnerability occurred because the backend directly resolved the user-controlled filePath variable against the application's upload directory without first validating whether the value was absolute or outside of the permitted directory. As a result, an attacker could submit values like /etc/passwd or encode ../ and retrieve data outside the given directory. The fix converted the upload directory to a normalized absolute path, rejected null, blank, invalid, and absolute input paths. Furthermore, the fix ensured the directory remained within the allowed permissions and invalid requests returned a HTTP 400 response instead of the filesystem contents after the fix. Targeted unit tests and testing confirmed invalid requests returned HTTP 400 responses and would not disclose the file as beforehand. This demonstrated that the download path traversal vulnerability was successfully fixed.

- **MySQL time-based SQL injection** was identified by OWASP ZAP as a high-risk, medium-confidence vulnerability in POST /api/v1/users/login. The vulnerability occurred because the login repository constructed a native SQL statement by simply concatenating email and password directly into the query, leaving leeway for malicious input containing elements such as quotes, SQL comments, or expressions such as SLEEP(15) to modify SQL statements shown explicitly during fuzzing. The fix replaced the vulnerable dynamically constructed SQL with a constant query that contains named parameters :email and :pasword, with supplied credentials now being supplied using the JPA's available Query.setParameter() method. Subsequently, the login service was updated to call the corrected repository method, and the ineffective browser-side SQL pattern detector and its localStorage flag were removed. The initial rudimentary approach was not effective as it recognized only a small collection of patterns, examined only the client-controlled input, and could be bypassed completely by sending requests directly to the API. The fix successfully remediated the issue and successfully improved application standing during SonarQube, ZAP, and targeted case testing, confirming that SQL injection payloads could no longer alter query execution or produce time-based delays.

Recommendations & Enhancements

To further enhance the security testing posture of the VEGA application beyond SonarQube and OWASP ZAP, the development team should integrate Software Composition Analysis (SCA) tools (such as OWASP Dependency-Check or Snyk) into the CI/CD pipeline. SCA tools automatically scan third-party npm and Maven dependencies for known Common Vulnerabilities and Exposures (CVEs), addressing supply chain risks that standard SAST and DAST scans may overlook.

Additionally, implementing Interactive Active Security Testing (IAST) or Runtime Application Self-Protection (RASP) mechanisms would provide continuous monitoring inside the application server during integration testing, catching complex logic flaws and authorization bypasses before reaching staging environments.

Conclusion

The combination of SAST and DAST methodologies provided a highly effective framework for securing the vega application. By defining source-code and anti-paterns with SonarQube and validating active exploit paths OWASP ZAP, the team successfully discovered and remediated critical vulnerabilities including SQL Injection, Path Traversal, and Cross-Domain Misconfigurations. The final post-fix analysis confirms a clean dynamic security profile, rendering the application robust and ready for secure production deployment.